

The Serotonin Operating System Manual

Volume 2: C/C++ Usermode Programming

Version: 0.4.0

February 27, 2026

Contents

1	Getting Started	2
1.1	Tutorial: Hello World	2
1.2	Example: Process Management	2
1.3	Example: Inter Process Communication	3
1.4	Example: SSE2 Acceleration	3
1.5	Example: Interacting With Devices	3
1.6	Example: Drawing GUIs	3
2	System Call Manual	3
2.1	exit	3
2.2	write	3
2.3	read	3
2.4	execve	3
3	Desktop Enviornment Programming	3

1 Getting Started

Serotonin is a Unix-like operating system. If you already have experience programming for other Unix-like operating systems (Linux, macOS), especially in C or C++, then you will probably find it easy to program for Serotonin. There does exist non-POSIX system call (lib5ht), but they are designed to be easier to interface with compared to typical Unix-like operating systems. Overall, if you just want to have some fun programming on an easy to learn yet powerful operating system, Serotonin is well suited for it.

1.1 Tutorial: Hello World

Prerequisites:

- A full clone of the Serotonin git repository with all tools built OR
- A copy of binaries, including all development tools.

The cross-compiler is especially important, as it is required for compiling programs for Serotonin on other operating systems. It is assumed that you know how to compile a program for Serotonin. If not, please read Volume 1: 3.1/3.2 for more guidance.

```
1 #include <stdio.h>
2 int main(void) {
3     printf("Hello, world!\n");
4     return 0;
5 }
```

You will notice that this is basically the same for almost all Unix-like operating systems, as Serotonin provides the Newlib C Standard Library.

1.2 Example: Process Management

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main(int argc, char **argv, char **envp) {
4     printf("I am the parent.\n");
5     pid_t fork_result = fork();
6     if (fork_result < 0) {
7         // fork failed
8         return 1;
9     } else if (fork_result == 0) {
10        printf("I am the child.\n");
11        execve("/bin/sh", {"bin/sh"}, envp);
12    } else if (fork_result > 0) {
13        printf("child pid: %d\n", fork_result);
14        int status;
15        waitpid(fork_result, &status, 0);
16    }
17    return 0;
18 }
```

1.3 Example: Error Handling

1.4 Example: Inter Process Communication

1.5 Example: SSE2 Acceleration

1.6 Example: Interacting With Devices

1.7 Example: Drawing GUIs

2 System Call Manual

2.1 `exit`

- Kernel version: 0.1.0+
- Usage: `_exit(int status)`

Terminates the process. This function does not return. This is called by the Serotonin runtime when returning from `main()`.

2.2 `write`

- Kernel version: 0.1.0+
- Usage: `write(int fd, const void* buf, size_t count)`

Write to a file descriptor. Returns the number of bytes written, or -1 indicating an error.

2.3 `read`

- Kernel version: 0.1.0+
- Usage: `read(int fd, void* buf, size_t count)`

Read from a file descriptor. Returns the number of bytes read, or -1 indicating an error.

2.4 `execve`

- Kernel version: 0.2.0+
- Usage: `execve(const char *name, char *const argv[], char *const envp[])`

Replaces the current program image with a new program. This function only returns on error.

3 Desktop Enviornment Programming